



BRAND · RECEIPT GATE · THE COMPLETE DOCUMENTATION · v3.9.0

One bounded task in.
One authorized attempt, portable handoff,
and independently verified acceptance out.

- BOUNDS
- SEALS
- HANDS OFF
- PROVES
- ACCEPTS

A contract-and-receipt system around agentic work. This is not “better prompting” — authority, execution, and acceptance are different jobs.

SOURCES · [N] README.md · VERSION · spec/task-spec-v3.md

THE QUESTION THIS DECK ANSWERS

When a worker says “done” — who is allowed to agree?

Not the worker. A prompt delegates intent, but it cannot bind scope, proof, or acceptance. Task-Spec makes those three decisions **explicit, sealed, and checkable** — before, during, and after every attempt.

Prompts can delegate work. They cannot define trusted done.

Model quality is real – but a chat prompt carries no binding scope, no sealed revision, and no independent verdict. “Done” arrives as a **worker claim**, not an accepted fact.

FAILURE · CHAT-BOUND

INTENT DRIFTS

The brief lives in chat. Scope moves while the worker runs, and “done” is reconstructed mid-execution – by the one party with an interest in finishing.

SCOPE=UNBOUND

FAILURE · WEAK WITNESS

PROOF OVERFITS

Tests pass on the patch and would have passed on the unpatched baseline. Green output that cannot discriminate change is not a verdict.

EVAL_NONDISCRIMINATING

CONTRACT · SEALED

A TASK-SPEC BINDS

Behavior, write surface, evals, budget, and acceptance travel as **one sealed revision**. Mutation breaks authorization; acceptance reruns the proof.

TASK_REVISION=sha256:...

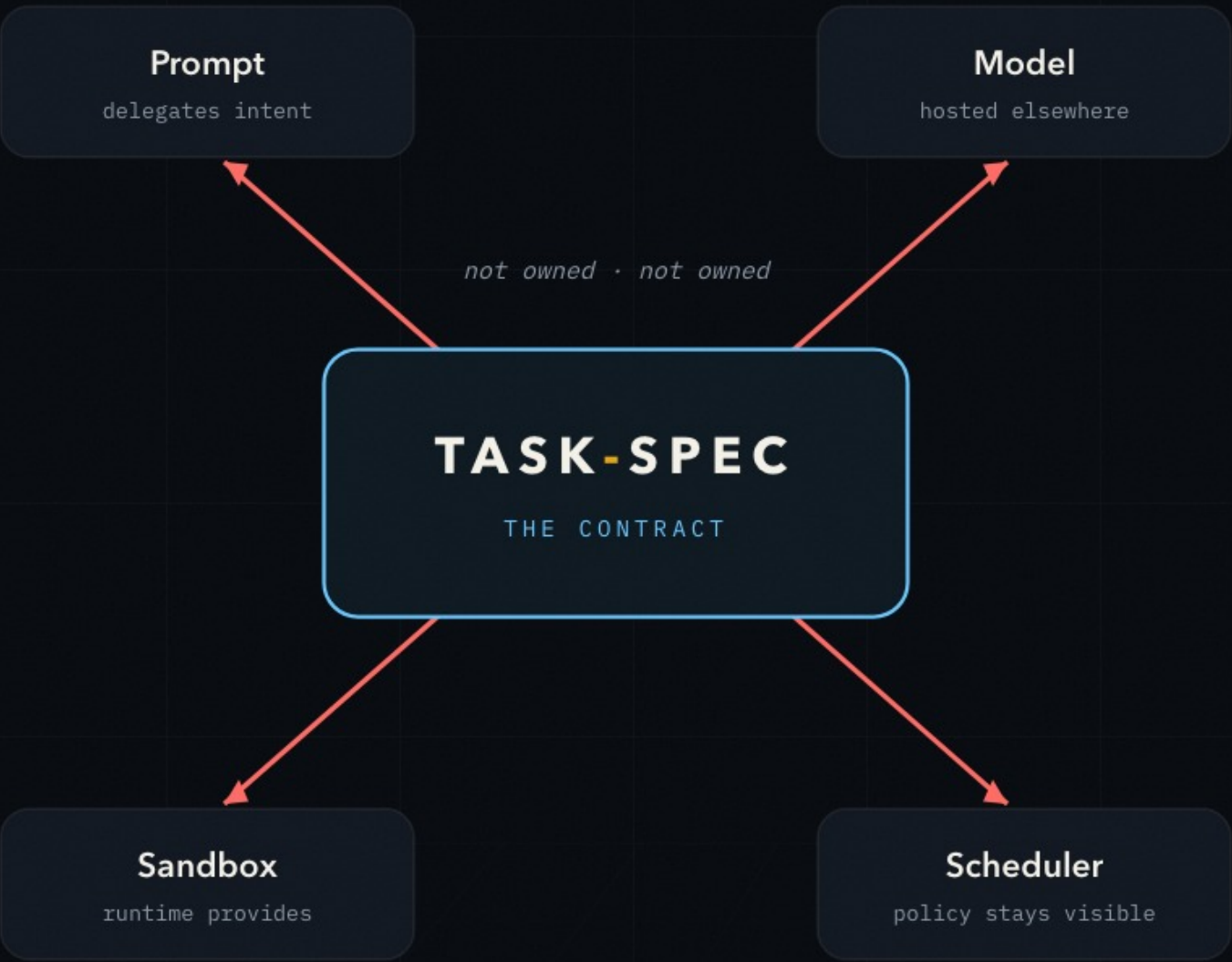
“A good agent can write the patch. A trustworthy system can explain why that exact patch was allowed to become done.”

– THE RECEIPT GATE · AUTHORITY SPLIT

SOURCES · [N] README.md · spec/task-spec-v3.md · cited evaluation research on overconfident completion claims

It is a contract, not another worker.

Task-Spec sits between intent and execution. It does not host the model, run the sandbox, or schedule silently – it **binds authority and proof** around whoever does.



authority lives in the sealed revision – not in the runner

TASK-SPEC OWNS

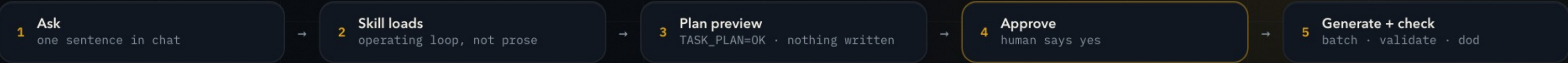
- bounded contract
- executable proof
- sealed authority
- portable handoff
- independent acceptance

scope + behavior + evals in one object
runnable evals, rerun at acceptance
TaskRevision + HMAC v3
TaskHandoff/v3 crosses harnesses
POST writes AcceptanceRecord/v1

TASK-SPEC DOES NOT OWN

- × model hosting
- × credentials
- × hostile-code sandbox
- × silent scheduling policy
- × universal semantic truth

You ask in chat. The skill drives the CLI. A task file appears.



chat – harness session

session

U

User – “Add exact-symbol search to the repository index.”

SKILL LOADED · TASK-SPEC

A

Agent – ran taskspec plan → TASK_PLAN=OK. One S-sized unit, writes src/search.py + tests/test_search.py. **Nothing is materialized yet.** Approve the topology?

U

User – “Approved.”

A

Agent – batch · validate · dod → TASK_BATCH=OK DOD=COMPLETE. Leaf written. signed_off: false, accepted: false – creation grants no authority.

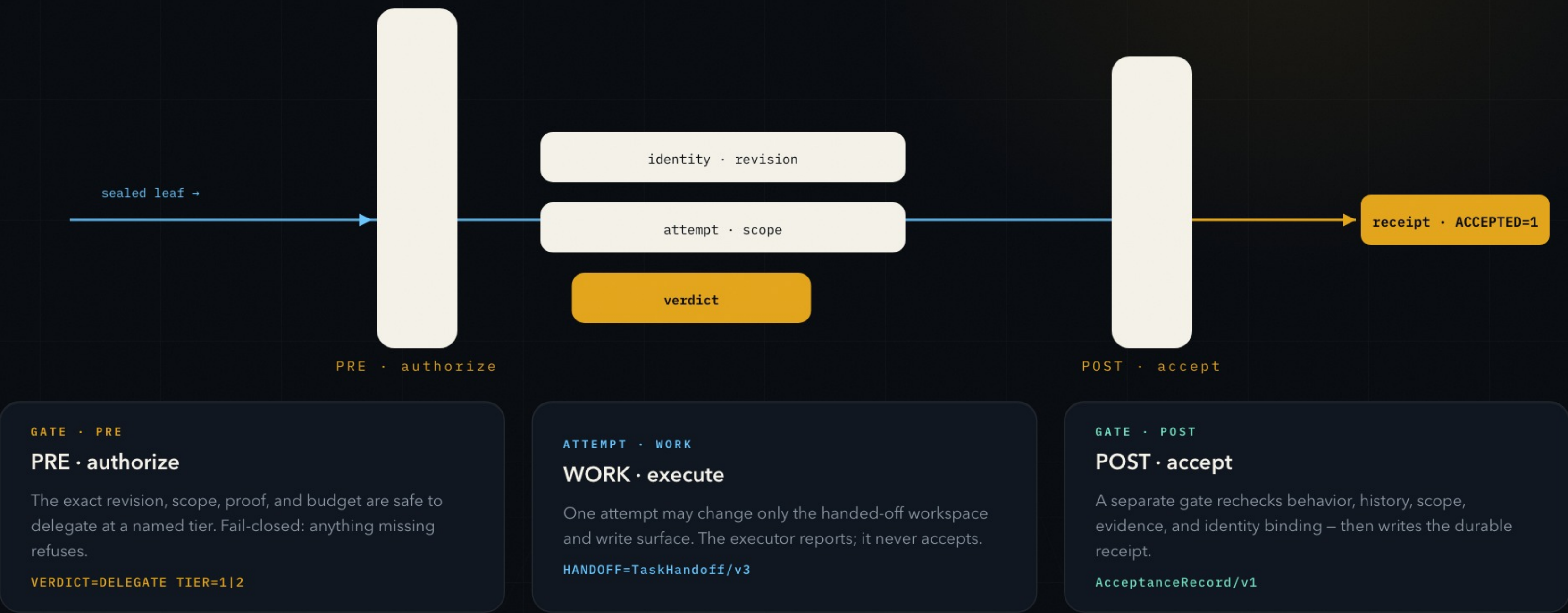
tasks/.plans/change.yaml – agent wrote

yaml

```
api_version: taskspec.dev/v1
kind: TaskPlan
approved: false
units:
  - id: T-20260826-index-search
    effort: S touches_paths: [src/search.py] creates_paths: [tests/test_search.py]
```


PRE says "may run." POST says "may close."

Authorization and acceptance are **independent events** with different owners. The brand mark is the mechanism: two posts, one receipt, a gold verdict line.



One contract, end to end. Nothing becomes done by narration.

Every step emits a machine-readable token. The two gold stamps are **human jobs**; the mint terminal state exists only after independent acceptance.



Human intent → TaskPlan/v1 → atomic leaf → PRE → TaskHandoff/v3 → attempt → POST → AcceptanceRecord/v1

WHY THE RAIL HOLDS

No single actor owns every decision. The author proposes, a human seals, the executor stays inside the write surface, and a separate acceptor reruns the proof.

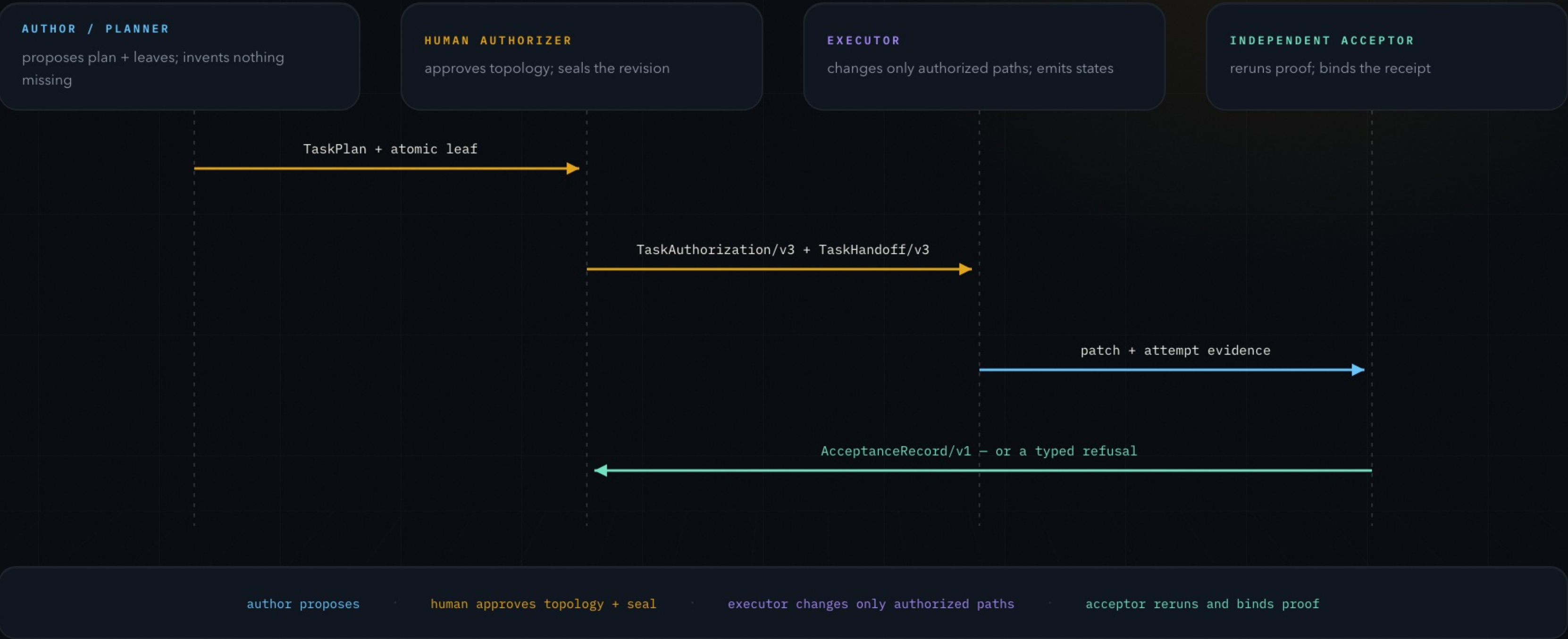
READ THE TOKENS

Each token is greppable and emitted by the CLI, not by the model: **TASK_PLAN=OK** is reviewable, **DOD=COMPLETE** is provable, **ACCEPTED=1** is settled.

SOURCES · [I] src/setup/demo.sh · bin/taskspec · [E] tests/test-demo.sh

Four roles. No self-issued authority.

Author, authorizer, executor, acceptor hold **different powers**. The rails below are the only things they exchange – typed objects, not trust.



SOURCES · [N] OPERATING.md · [I] src/author/ · src/gate/ · src/dispatch/ · src/accept/

A leaf has frontmatter and six zones.

Identity and lifecycle live above the fold; executable meaning lives below. One Markdown object carries **everything a gate, a worker, or an acceptor needs**.

tasks/T-leaf.md – frontmatter cap

yaml

```
id: T-...  status: ready  format_version: 3  effort: S  depends_on: []
touches_paths: [...]  creates_paths: [...]  signed_off: false  accepted: false
```

Z1	Intent	why this exists, in the owner’s terms
Z2	Behavior	GIVEN / WHEN / THEN observables
Z3	Contract	write surface = touches U creates
Z4	Guardrails	anti-patterns · stop · reversal
Z5	Operations	validation card · evals · retry
Z6	Reversal & Runtime	rollback · observability · assumptions

READ TOP-DOWN

The cap answers **who, where, and what state**: stable identity, dependency edges, write surface, backend, and the two authority bits. The zones below answer **what must become true, and how we will know**.

ONE OBJECT, MANY READERS

PRE reads it to decide delegation. The executor reads it as the whole job. POST reads it as the acceptance contract. A human reads it as documentation. **Nobody re-explains it in chat.**

NAMING NOTE

“Six zones” is teaching vocabulary; the normative v3 structure is the source of truth. Where docs and spec differ in naming, **the spec wins**.

[N] spec/task-spec-v3.md · [D] docs/concepts/six-zones.md

SOURCES · [N] spec/task-spec-v3.md · [D, historical naming] docs/concepts/six-zones.md

Identity is explicit. Lifecycle is constrained.

Accountability is **data, not chat history**: every leaf carries identity, dependency, write, execution, and authority fields in a schema-validated cap.

tasks/T-20260826-repository-search.md yml

```
id: T-20260826-repository-search
status: ready
format_version: 3
profile: standard
effort: S
depends_on: []
touches_paths: [src/search.py]
creates_paths: [tests/test_search.py]
execution_backend: any
signed_off: false
accepted: false
```

IDENTITY	id + format_version – a stable name that survives renames of everything else.
GRAPH	depends_on – edges the derived graph uses for waves and readiness.
WRITE	touches u creates – the complete set of paths this attempt may change.
RUNTIME	effort + execution_backend + profile – size gate, routing hint, proof depth.
AUTHORITY	signed_off / accepted – flipped only by PRE and POST, never by authoring.

XS to L execute. XL and XXL only compose.

Large work must be **decomposed before it can be delegated**. Size is a breadth gate: it caps the write surface and decides whether a card is executable at all.

XS · LEAF

1 path

DIRECT HANDOFF

S · LEAF

2 paths

DIRECT HANDOFF

M · LEAF

3 paths

DIRECT HANDOFF

L · LONG HORIZON

5 paths

CAPABLE BACKEND ONLY

XL / XXL · NODE

0 writes – owns children

child S

child S

child M

HANDOFF REFUSED

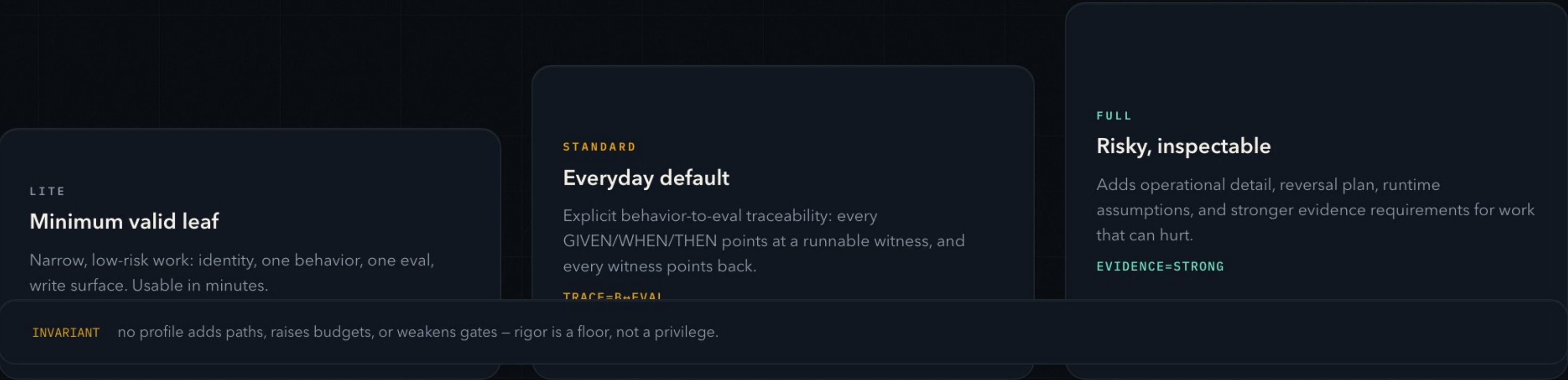
SIZE	ROLE	MAX WRITE-SURFACE ITEMS	DIRECT HANDOFF
XS	leaf	1	yes
S	leaf	2	yes
M	leaf	3	yes
L	long-horizon leaf	5	only on a capable backend
XL / XXL	composition node	0	refused

WHY an executor that cannot hold the whole job in one bounded attempt should not receive one – decomposition is authorization hygiene.

SOURCES · [N] spec/task-spec-v3.md · [I/E] src/gate/validate-task-spec.sh · tests/test-effort-sizing.sh

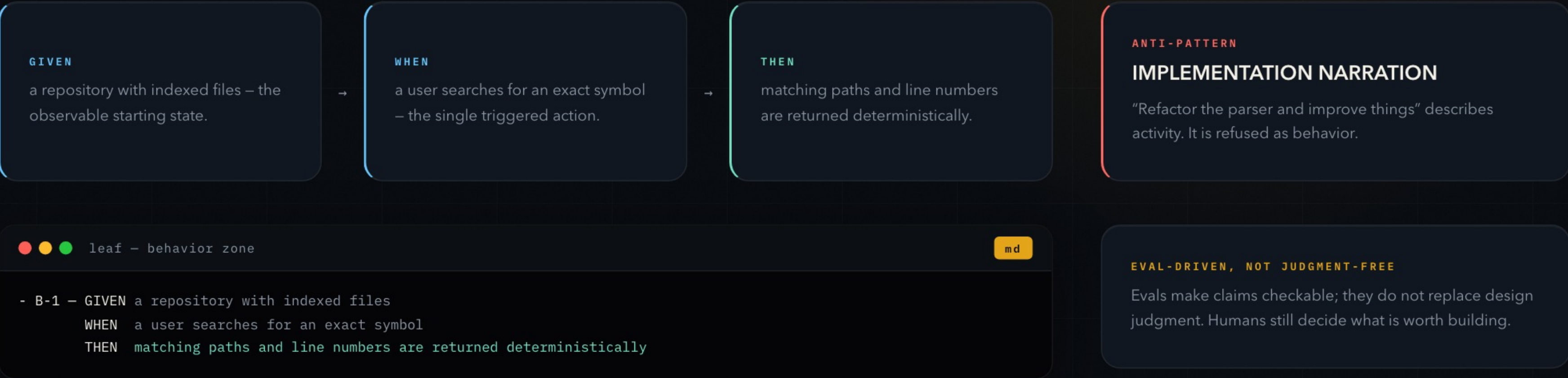
Lite, standard, full: proof scales with risk.

A tiny leaf should stay usable; a risky leaf should stay inspectable. Profile changes **required depth** – it never expands write authority.



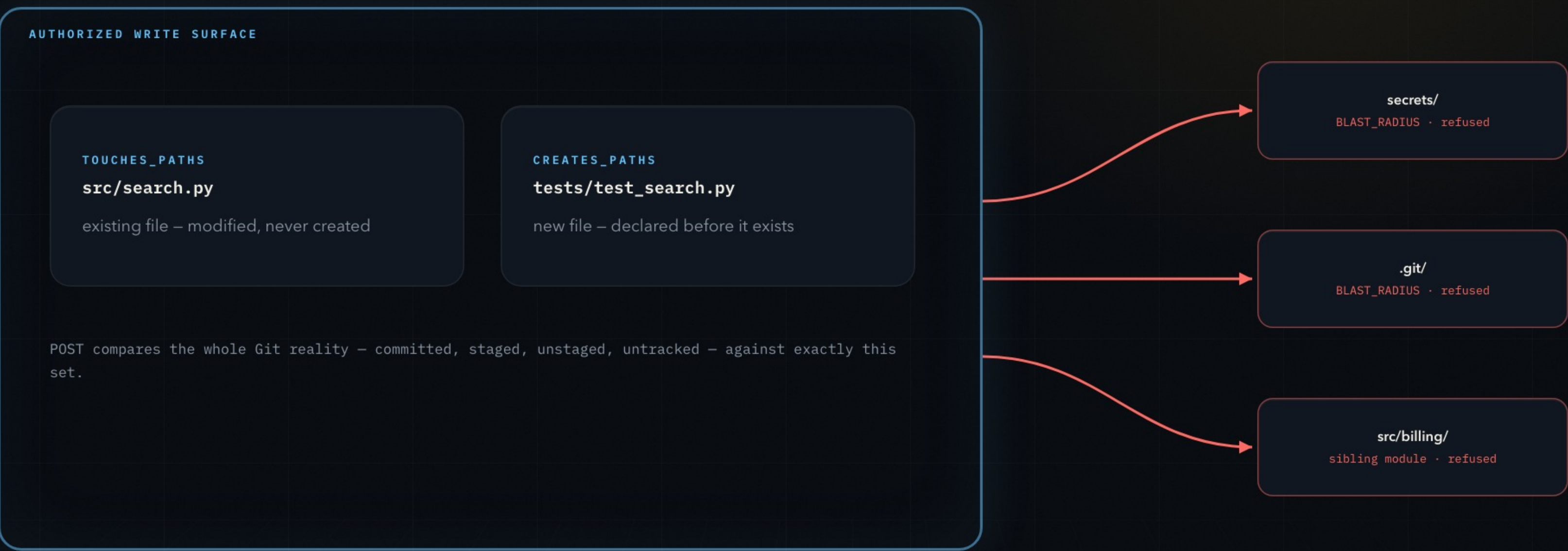
Write the behavior before the patch.

Success is an **observable state transition**, not a to-do list. If a behavior cannot be witnessed by an eval, it is not contract – it is hope.



The write surface is a set, not a vibe.

`write_surface = touches_paths U creates_paths` – existing files go in touches, new files in creates, **everything else is outside authority**.



Refusal is part of the contract.

Anti-patterns, do-not-touch paths, retry policy, stop conditions, reversal, and runtime assumptions are **first-class zones** – failure is a designed outcome, not an agent surprise.

PREVENT

Name the forbidden

Explicit anti-patterns and do-not-touch boundaries: no silent refactors, no dependency additions, no edits outside the moat.

DO_NOT_TOUCH=[...]

STOP

Bound the attempt

Iteration budget, a no-progress circuit breaker, and defined terminal-failure behavior. Stopping early is success, not shame.

MAX_ITER=12 · NO_PROGRESS=3

RECOVER

Plan the way back

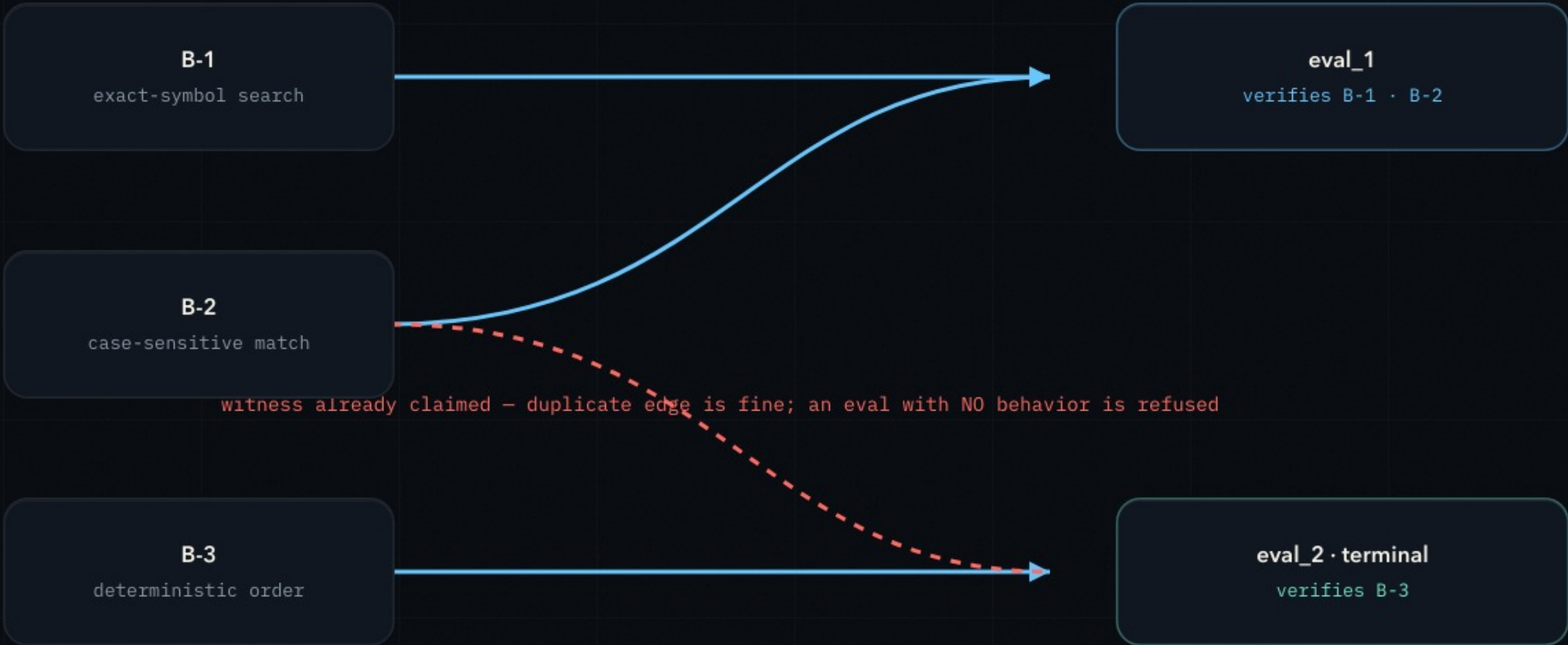
Rollback/reversal path, observability hooks, and runtime assumptions – so a parked attempt leaves context, not wreckage.

PARK_WITH_CONTEXT

RED → GOLD → MINT the same attempt carries its own prevention, its own budget, and its own exit.

No orphan behavior. No unexplained eval.

Every behavior needs a witness; every eval needs a reason. The bipartite graph below is the load-bearing rule behind **DOD=COMPLETE**.



THE RULE

DoD walks the matrix both directions: a behavior with no eval is a **DOD_GAPS** refusal; an eval that verifies nothing is an unexplained cost and is flagged the same way.

DOD=COMPLETE ⇔ **matrix full**

TERMINAL MEANS DECISIVE

A terminal eval ends the attempt when it passes: it is the suite that must fail on the unpatched baseline and pass on the patch.

terminal: true

The Validation Card is the execution contract for proof.

Eval metadata plus the Agent Contract make a leaf **portable across harnesses**: any conformant runner knows what to run, how often, and what it may emit.

leaf – operations zone

yaml

```
success_criteria:
  - id: eval_1  runnable: bash  check_type: deterministic
    verifies: [B-1, B-2]  terminal: true
retry_policy:
  max_iterations: 12  circuit_breaker_no_progress: 3
  on_terminal_failure: park_with_context
agent_contract: { version: 2, read: [intent, behavior, contract, guardrails],
                  produce: [code, tests], emit: [pass, fail, retry_with_reason, parked_with_
```

RUN • RETRY • EMIT

The runner executes declared evals at agent cadence, retries inside the budget, and emits one of four machine states. **No free-form “almost done.”**

RUNTIME-NEUTRAL

Codex, Claude Code, Kimi, or a bare shell all read the same card. The contract names the checks; the harness only supplies compute.

WEAKNESS IS LABELED

check_type says what kind of judgment an eval is. Existence-only checks are allowed but annotated – never mistaken for behavioral proof.

One leaf, fully assembled.

Identity, behavior, write authority, evals, and lifecycle travel together. Sanitized from a real accepted dogfood leaf – **digests redacted**.

tasks/done/T-20260815-acceptance-finalized-contract.md (sanitized) md

```
id: T-20260815-acceptance-finalized  effort: M  profile: standard
touches_paths: [src/accept/finalize.py]  creates_paths: [tests/test_finalize.py]
B-1 – GIVEN an attempt with passing terminal evals WHEN accept stamps
      THEN record, envelope, and metrics are written atomically
eval_1: bash tests/test_finalize.py  verifies: [B-1]  terminal: true
# Exit Check: behavior ↔ eval matrix complete, budgets declared
signed_off: true  authorization: hmac-sha256-v3:...  tier: 1
accepted: true  acceptance_record: sha256:...  attempt_id: uuid
```



Intent becomes a preview before it becomes files.

taskspec plan validates a proposed topology **without materializing tasks** – architecture is reviewed as a picture, not discovered as a surprise.

tasks/.plans/repository-search.yaml yaml

```
api_version: taskspec.dev/v1
kind: TaskPlan
approved: false
metadata:
  name: repository-search
units:
  - id: T-20260826-index-search
    effort: S
    depends_on: []
    touches_paths: [src/search.py]
    creates_paths: [tests/test_search.py]
```

UNITS

each proposed leaf with size and backend hint

EDGES

dependency arrows, checked for cycles and dangling refs

PATHS

write sets compared pairwise for conflicts

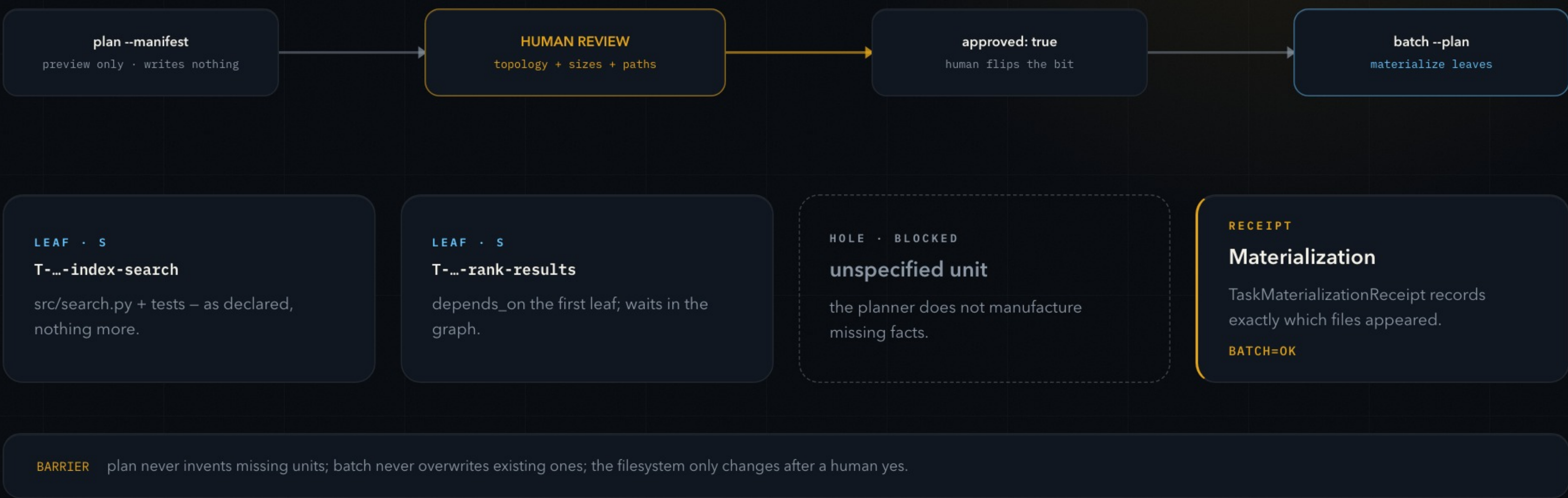
CONFLICTS

same-file collisions surfaced **before** anything exists

TASK_PLAN=OK means structurally reviewable – **not** human-approved.

Approval is the topology barrier. No approval, no task files.

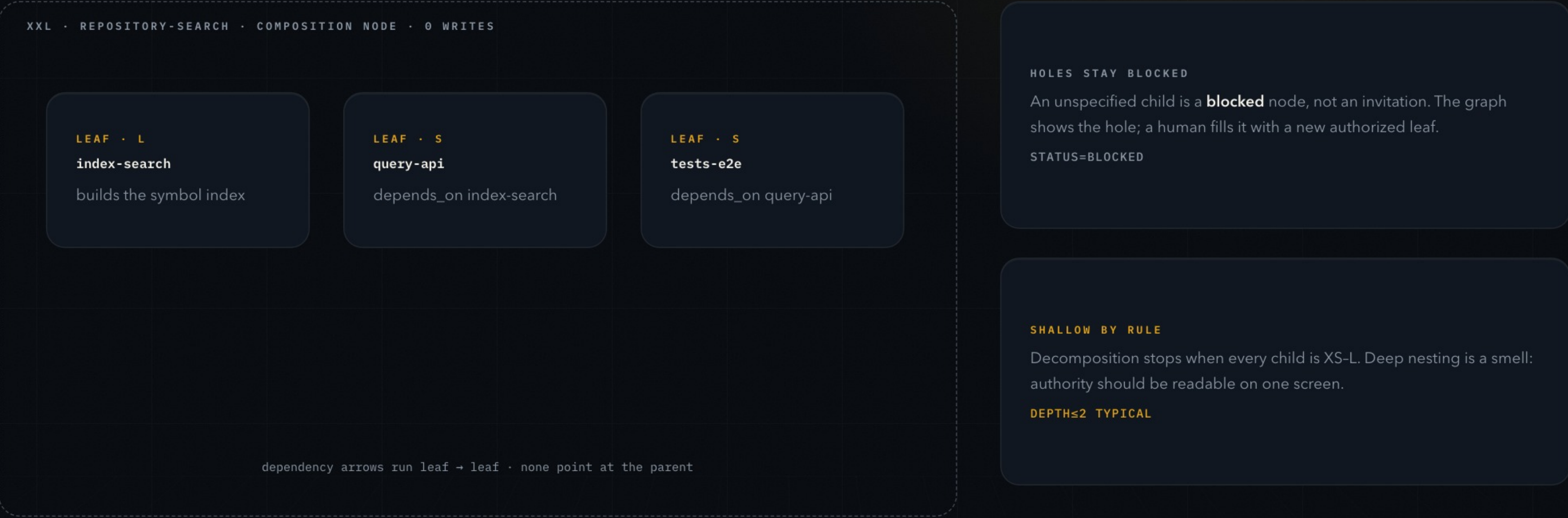
The model may propose the graph. A human decides whether it becomes work. batch is non-clobbering and materializes only the declared, approved manifest.



SOURCES · [I] src/author/taskplan.py · src/author/batch-generate.sh · [N] spec/schemas/task-materialization-receipt.schema.json

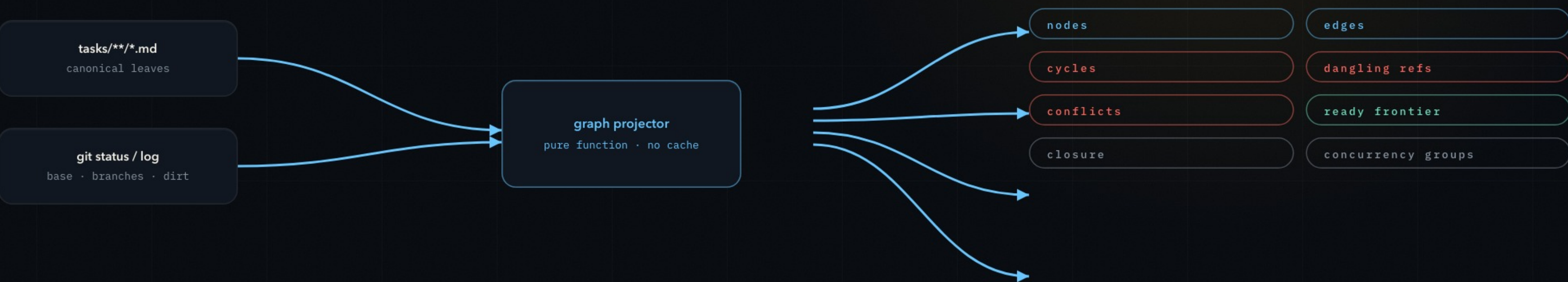
Composition organizes; leaves execute.

XL/XXL nodes own children and **zero writes**; leaves own behavior and paths. No arrow may ever make a parent executable – “epic as task” is refused.



The graph is derived, not a second source of truth.

TaskGraphView is a deterministic, read-only projection of **Markdown + Git**. Delete it and the next command rebuilds it identically.



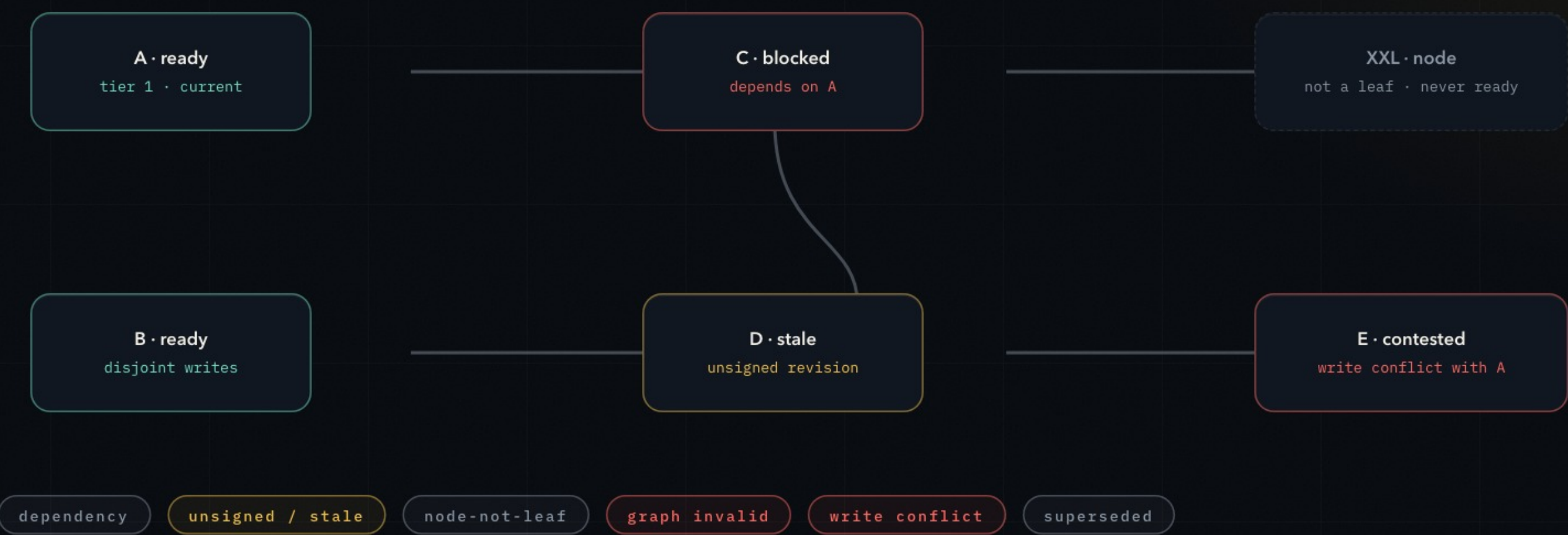
`taskspec graph --check` · `taskspec graph --mermaid` – same input, same output, every time.

WHY IT MATTERS Schedulers, cockpits, and humans may **read** the projection freely – but authority stays in the leaf files. The view can be wrong; the contract cannot be silently rewritten.

SOURCES · [N] `spec/schemas/task-graph-view.schema.json` · [I] `src/graph/task_graph.py` · [D] `docs/reference/task-graph-view.md`

Ready means more than status: ready.

A runnable frontier requires a valid leaf, satisfied dependencies, current revision, acceptable tier, and no blocking collision – **five conditions, all explicit.**

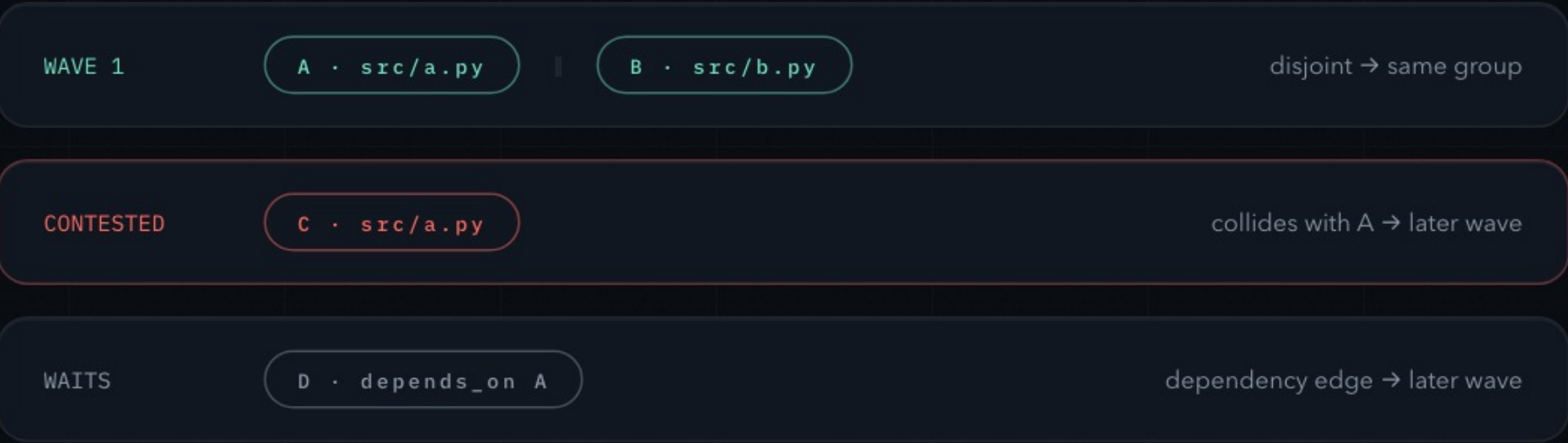


FRONTIER only A and B are eligible now – and eligibility is recomputed, never remembered.

SOURCES · [I] src/graph/task_graph.py · src/backlog/list-ready.sh · [E] tests/test-backlog-analysis.sh

Parallel is a write-set decision.

Deterministic concurrency groups contain only **write-disjoint** leaves. Two ready leaves may still collide on disk – the graph says so before workers do.



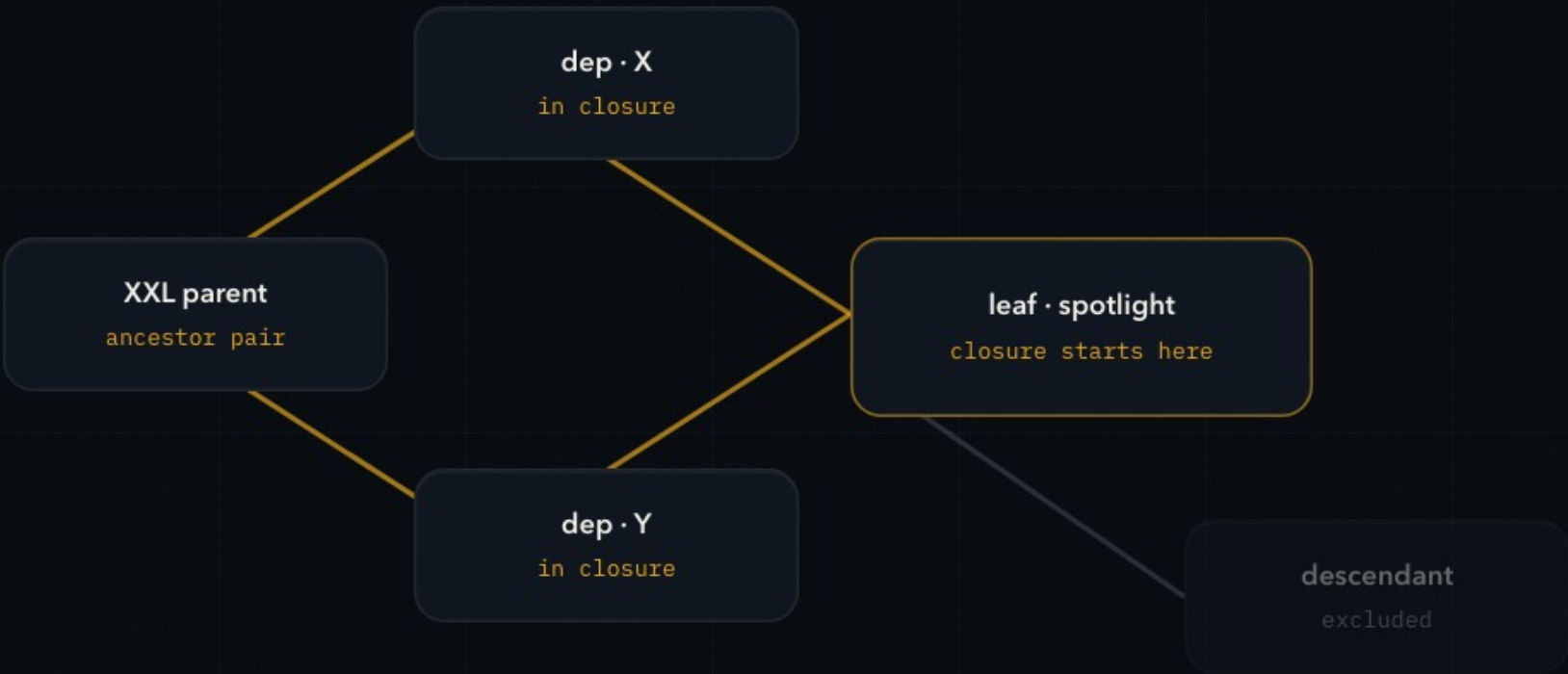
RELATIONSHIP	GRAPH RESULT
disjoint write sets	same concurrency group
same existing file	write conflict
same new file	dual-create error
dependency edge	later wave

NO NEGOTIATION AT RUNTIME

Workers never lock, merge, or race. The grouping is computed from declared paths – **before any attempt exists.**

The closure freezes what this leaf depends on.

Closure = the leaf + transitive dependencies + composition ancestors, as **TaskRevision pairs**. Handoff carries its digest; drift later is detectable, never silent.



WHAT THE DIGEST BINDS

Every ancestor and dependency enters as **(task_id, revision)**. Edit any of them and the closure digest changes – the old handoff goes stale at the next check.

```
closure_digest = sha256(ordered task_id + revision pairs)
```

WHY

An attempt must not execute against quietly-rewritten context.
CLOSURE_DRIFT is a typed refusal, not a heuristic.

SOURCES · [N] spec/schemas/task-handoff.schema.json · [I] src/graph/task_graph.py · src/dispatch/handoff.py

Replanning creates new authority.

A material change becomes a **successor task** and a **new authorization decision**. History is parked, never rewritten; dependents are re-pointed by a human.



⊘

REFUSED SHORTCUT

"silent downstream rewrite" – editing a signed leaf and hoping dependents won't notice breaks the seal and the audit trail.

THREE CLASSES OF CHANGE

Tiny pre-execution correction → **amend + reseal**. Materially different or partially executed → **successor with supersedes**. Interrupted mutation → **park with context**.

DoD checks the contract before it checks the code.

DOD=COMPLETE means the behavior/eval/Exit-Check matrix is complete – **not** that the feature is implemented. Provability first, implementation second.

BEHAVIOR ↓ · EVAL →	EVAL_1	EVAL_2	EXIT CHECK
B-1	●	○	●
B-2	●	○	●
B-3	○	●	●
B-4 · orphan	○	○	×

FULL MATRIX

Every row has a witness, every column a reason, Exit Check signed. The token reads DOD=COMPLETE and PRE may continue.

ONE MISSING WITNESS

B-4 has no eval. The flip is mechanical, not judgmental: DOD=GAPS – the spec is not yet provable, so it is not yet delegable.

READ CAREFULLY “the task is well-specified” ≠ “the work is done”. DoD is a gate on the contract, and acceptance later reruns the same witnesses.

PRE is a fail-closed pipeline.

Size, structure, traceability, shell integrity, and runnable evals must all survive **before** stamping. `gate --stamp` is the last step, not a signature shortcut.



VERDICT IS TYPED

VERDICT=DELEGATE TIER=1|2 or a named refusal – never silence. Tier 2 means supervised, and says so on the record.

WHAT THE STAMP SIGNS

Not the filename – the **canonical TaskRevision**. Any authority-bearing edit after the stamp breaks verification at the next gate.

TaskRevision is the thing being authorized.

Mutable reporting fields are excluded; authority-bearing fields – including unknown future fields – are sealed by default.



what enters the digest

text

```
TaskRevision/v1 = sha256(  
  canonical task identity  
  + graph edges  
  + write surface  
  + behavior and evals  
  + budgets and execution policy  
  + all unknown authority-bearing fields  
)
```

WHY DEFAULT-SEALED MATTERS

Scope, dependency, budget, agent, or backend edits **break the seal** – while allowed status/reporting updates do not. Future fields cannot sneak in unsealed: unknown authority-bearing content is hashed, not ignored.

SEAL=DEFAULT-CLOSED

Three tiers, three operating policies.

The same digest can be trusted at three very different levels. **Tier 1 eligible** – never “cryptographically safe” – is the honest phrasing.

TIER 1

Verified HMAC v3, current revision

Unsupervised dispatch eligible – if every other gate passes. The seal says tamper-evident under a shared key; the gates say the rest.

UNSUPERVISED ELIGIBLE

TIER 2

Key unavailable · legacy · narrower proof

Supervised only. Acceptance demands an explicit supervisor, reason, and override on the record.

SUPERVISED ONLY

TIER 3

Malformed · wrong key · revision mismatch

Hard refusal at PRE and again at POST. There is no override path into execution.

REFUSE

FOOTER LAW HMAC is shared-key tamper evidence – **not** identity, isolation, or semantic truth.

SOURCES · [N] docs/concepts/signed-off.md · [I] src/security/stamp.py · src/gate/safe-to-delegate.sh · [E] tests/test-hmac-envelope.sh

Change authority, break the seal.

Authority mutation and lifecycle reporting are **intentionally different classes**. The matrix turns cryptographic language into operational intuition.

MUTATION	RESULT
widen touches_paths	Tier 3 / re-authorize
change dependency	Tier 3 / re-authorize
raise iteration budget	Tier 3 / re-authorize
change agent / backend	Tier 3 / re-authorize
update allowed status field	seal remains valid
write tracker projection	seal remains valid

INTUITION If the change would alter **what a human approved**, it needs a human again. If it only reports progress, the receipt trail absorbs it without breaking authority.

SOURCES · [E] tests/test-hmac-envelope.sh · [I] src/security/task_revision.py

TaskHandoff/v3 is the pickup packet.

Every executor receives the same revision, base, closure, scope, budget, and acceptance route. **No signing key, evaluator key, holdout command, or credential enters the packet.**

.taskspec/handoffs/attempt.json

json

```
{
  "contract": "TaskHandoff/v3",
  "task_id": "T-20260826-index-search",
  "task_revision": "sha256:...",
  "authorization": {"tier": 1, "ref": "hmac-sha256-v3:..."},
  "attempt_id": "uuid",
  "workspace": "/absolute/worktree",
  "base_commit": "<immutable commit>",
  "closure_digest": "sha256:...",
  "write_surface": ["src/search.py", "tests/test_search.py"],
  "budget_iterations": 12,
  "execution_backend": "any"
}
```

PIN 1

task_revision – the sealed authority digest

PIN 2

authorization – tier + HMAC ref

PIN 3

attempt_id – one execution, one subject

PIN 4

workspace – absolute handed-off worktree

PIN 5

base_commit – immutable ancestor pin

PIN 6

closure_digest – frozen dependency context

PIN 7

write_surface – the moat, carried

PIN 8

budgets + backend – bounded execution

Same contract, different harness.

Codex, Claude Code, Grok Build, Cursor, Kimi – any conformant executor consumes the same leaf and handoff. The installed skill helps each harness find the CLI; **it does not replace the gates.**



Conformance is cumulative.

Executor capability is measured against explicit pickup, execution, verification, and termination obligations – **not against vibes**.

L0 · READS

Parses the contract

Reads the leaf, preserves identity, and reports states honestly. The floor for any harness that claims support.

L1 · EXECUTES

Stays inside scope and budget

Runs within the declared write surface and iteration budget; emits machine states: pass, fail, retry_with_reason, parked_with_context.

L2 · PROVES

Returns verifiable evidence

Produces evidence compatible with independent acceptance – so POST can rebind the result to the exact authorized attempt.

BOUNDARY conformance of the bundled reference path is evidence – **not** ecosystem-wide certification.

SOURCES · [N] spec/conformance/ · [D] docs/concepts/conformance-levels.md · docs/concepts/agent-contract.md

Evals run at agent cadence.

Deterministic evals give a bounded feedback loop inside one attempt: change → eval → pass / retry / park – bounded by iteration and no-progress counters.



SOURCES · [I] src/gate/run-task-spec.sh · [D] docs/patterns/runnable-bash-evals.md

evals – one behavioral, one terminal

```
# behavioral: fails on baseline, passes on patch
bash -c 'grep -q "def search_exact" src/search.py && pytest tests/test_search.py -q'
# terminal suite
make test-search # exit 0 ends the attempt
```

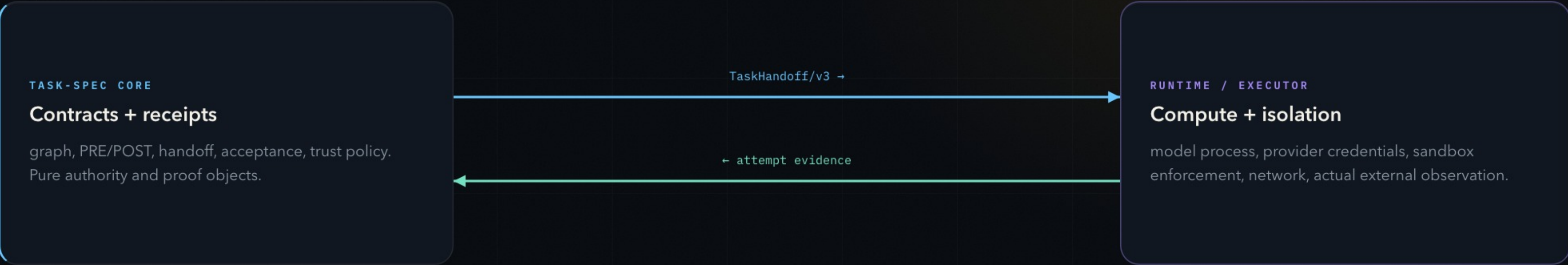
WEAKNESS, LABELED

Existence-only checks are weak. Behavioral probes should **fail on the unpatched baseline and pass on the patch** – green that cannot discriminate is annotated, never celebrated.

REMEMBER a passing attempt is a **candidate**. POST decides whether it can close.

The core boundary stays narrow.

Task-Spec produces authority and acceptance contracts; external runtimes provide model execution, credentials, and isolation. **The seam is the handoff.**



TASK-SPEC CORE	RUNTIME / EXECUTOR
contract + graph	model process
PRE authorization	provider credentials
handoff	sandbox enforcement
eval / POST acceptance	compute and network
receipts / trust policy	actual external observation

POST is a second gate, not a replay button.

Acceptance **rebinds the patch and evidence to the exact authorized attempt** – handoff, base, closure, scope, and integrity are all rechecked, not assumed.



INDEPENDENT MEANS SEPARATE

Different process, different inputs reconstruction: POST reruns evals itself and rebuilds the Git reality itself. The executor’s report is **input, not evidence**.

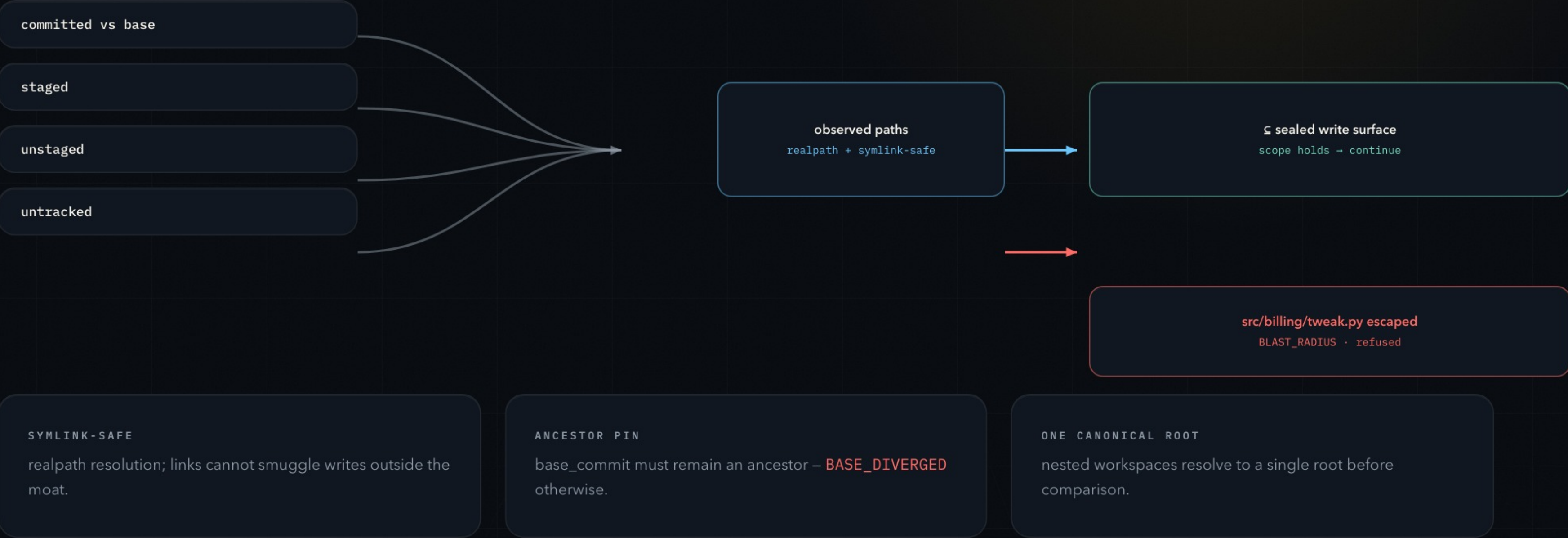
OUTPUT IS DURABLE

All checks green → [AcceptanceRecord/v1](#). Any red → a typed refusal on the record. Both outcomes are inspectable forever.

SOURCES · [I] `src/accept/accept-task.sh` · `src/accept/preflight.py` · `src/accept/finalize.py`

Blast radius uses the whole Git reality.

A clean-looking patch view cannot hide out-of-scope changes. POST merges **four Git layers** into one observed-path set and compares it with the sealed write surface.



SOURCES · [I] src/accept/preflight.py · src/lib/workspace.py · [E] tests/test-v381-workspace.sh · tests/test-v38-hardening.sh

Gold sanity asks whether the eval can discriminate.

The same eval should **fail on the unpatched baseline** and **pass on the candidate**. If baseline also passes, green proves nothing.

eval_1 @ BASELINE (unpatched)

fail

```
$ bash evals/eval_1.sh
- assert search_exact returns ["src/a.py", 12]
FAILED · 1 assertion
exit 1 – exactly what we want pre-patch
```

discriminates
gold sanity OK

eval_1 @ PATCH (candidate)

pass

```
$ bash evals/eval_1.sh
+ assert search_exact returns ["src/a.py", 12]
PASSED · 1 assertion
exit 0 – the change is observable
```

EVAL_NONDISCRIMINATING baseline also passes → the eval is a rubber stamp. Refused at POST under --gold-sanity.

HONEST SCOPE
Gold sanity is **opt-in** and raises confidence; it does not create universal semantic truth. A discriminating eval can still be a weak eval.

AcceptanceRecord/v1 is the durable verdict.

Task, revision, authorization, attempt, acceptor, time, and digest are recorded **together, atomically**. Exact retries are idempotent; conflicting retries refuse.

ACCEPTANCE RECORD · RECEIPT

task_id – T-20260826-index-search

task_revision – sha256:...

authorization_ref – hmac-sha256-v3:...

attempt_id – uuid

accepted_by – independent-reviewer

accepted_at – 2026-08-26T...Z

record_digest – sha256:...

ACCEPTED · SETTLED

ATOMIC FINALIZE

finalize writes the attempt record, the task acceptance envelope, and the metrics projection **as one transaction** – half-written acceptance does not exist.

RETRY SEMANTICS

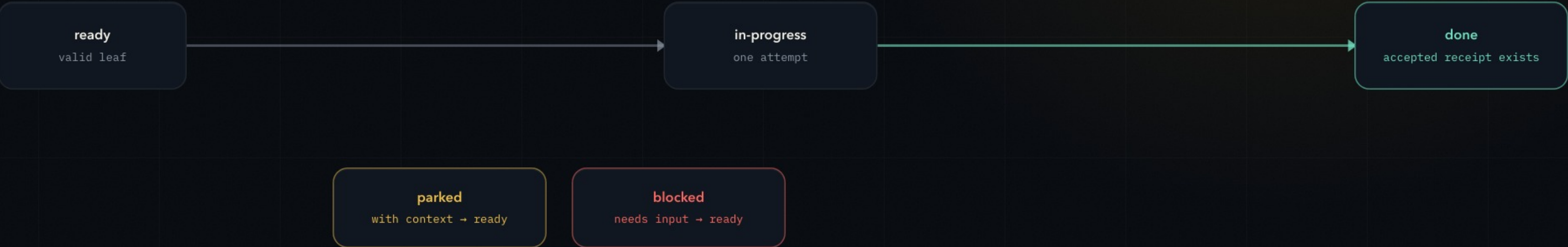
Same inputs → same receipt (idempotent). Different result for the same attempt → **refused**, because conflicting truths are tamper-shaped.

ONLY THIS RECEIPT

allows the lifecycle to reach **status: done**. Worker emotion is not a lifecycle state.

Done is a settled state, not a worker emotion.

status: done requires accepted: true. There is no status: sealed – sealing is authorization metadata, not backlog state.



TRANSITIONS ARE VALIDATED

The transition tool checks the status enum and enforces the acceptance condition on entry to done. This slide shows the common route, not an exhaustive edge whitelist.

AUTHORIZATION ≠ STATE

signed_off travels in the envelope; status travels in the backlog. Keeping them separate is why a parked task can still be sealed, and a running task never is “done”.

v3 is the default. v4 is the evidence upgrade.

Formats 1-4 remain readable; v4 adds stronger policies **without changing the core authority split**. v3 is not deprecated – it is the floor.

V3 · AUTHORIZING DEFAULT

- atomic leaf and six zones
- behavior ↔ eval traceability
- TaskRevision + HMAC
- handoff and acceptance
- deterministic eval path

V4 · OPT-IN ADDS

- explicit evaluation policy
- environment contract
- evaluator identity / trust
- sealed authoring evidence refs
- attempt-bound external receipts

EVOLUTION RULE new surfaces strengthen evidence; they never move the authority line between human, worker, and acceptor.

Four evaluation policies, four trust questions.

Deterministic, holdout, graded, and human evidence are **not interchangeable** – each answers a different question, and the receipt says which one was asked.

DETERMINISTIC

Reproducible assertion

Did a reproducible assertion pass? The everyday floor – runnable by anyone, identical twice.

check_type: deterministic

HOLDOUT

Private sealed cases

Did private cases pass without ever entering the worker context? Overfitting dies here.

sealed · hidden

GRADED

Bound rubric

Did an external evaluator meet a bound rubric and threshold? Judgment, but scoped and signed.

Ed25519 evaluator

HUMAN

Named approval

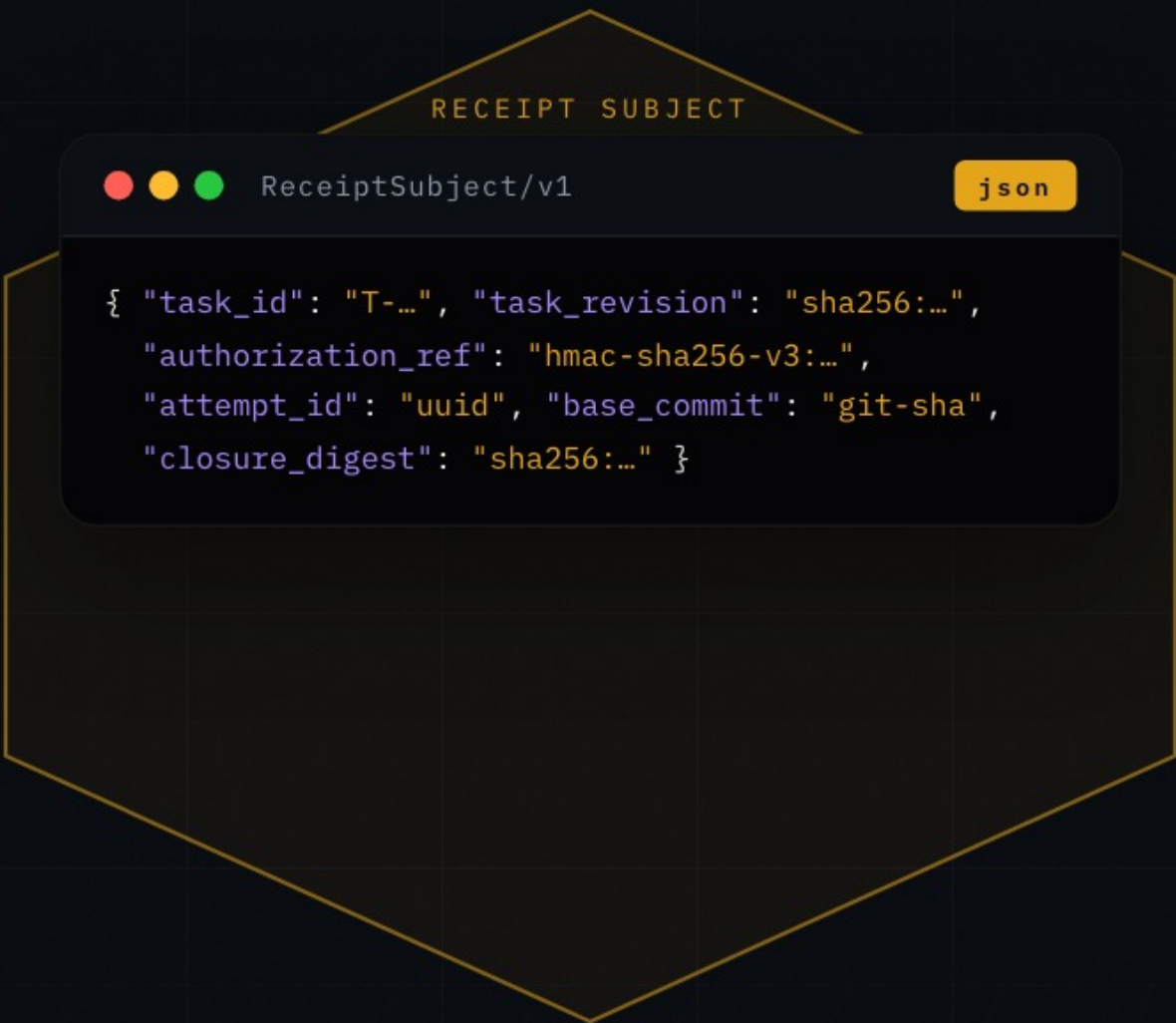
Did the named human approve the bound subject? The irreducible case, on the record.

human receipt

BINDING every non-deterministic verdict must bind to the same subject – next slide explains the six things that must match.

ReceiptSubject/v1 prevents evidence replay.

External evidence is valid only for **this** task, revision, authorization, attempt, base, and closure context. A valid signature on the wrong attempt still fails.



REPLAY, REFUSED

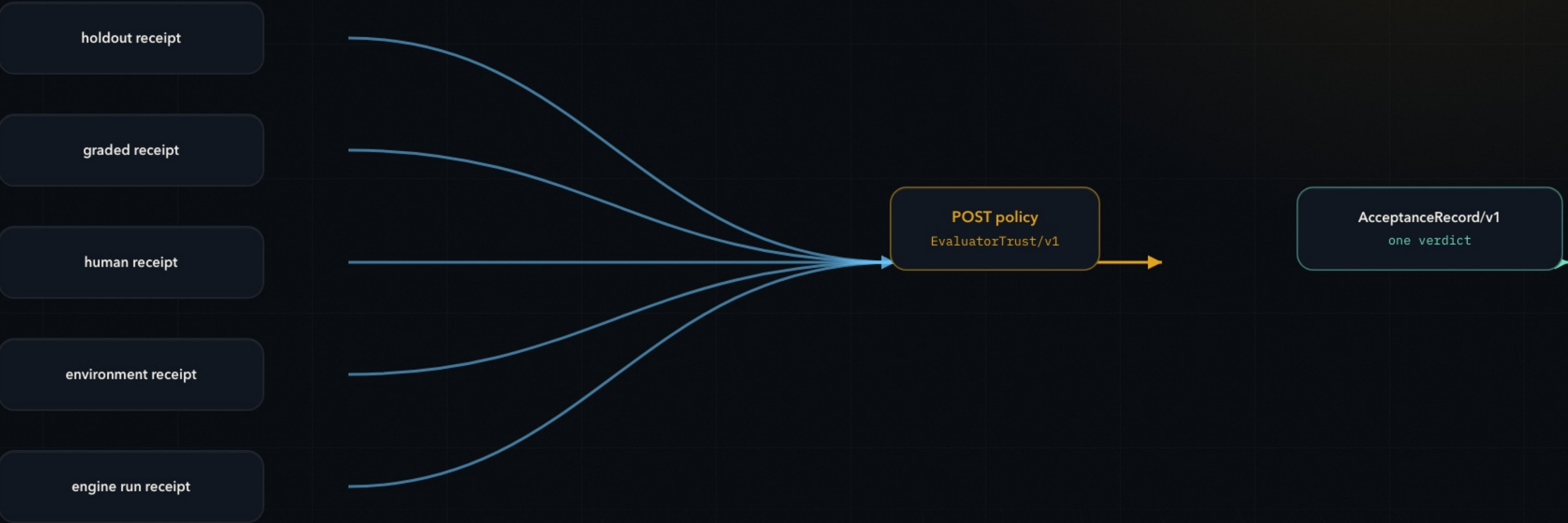
Copy a genuine holdout receipt from attempt A into attempt B and POST rejects it: the subject differs. Evidence is **attempt-bound**, exactly like the acceptance record.

CONTRIBUTE, DON'T USURP

Different observers can now contribute evidence – holdout, graded, human, environment – **without becoming the task authority**. POST weighs them under evaluator trust policy.

Stronger evidence fans in; authority does not fan out.

Five receipt types converge into POST under evaluator trust policy. The task authorization rail remains **separate and unchanged** – evidence informs acceptance; it never issues it.



SIGNATURE NOTE portable or human-authorized non-deterministic receipts require external Ed25519 trust – never the worker’s self-report.

Proof is scoped; risk remains explicit.

Controls eliminate **named** failure modes; they do not eliminate judgment. Every control on the left ships with its residual risk on the right.

CONTROL	RESIDUAL RISK
HMAC v3	a shared-key holder can authorize bad work
behavior/eval traceability	the behavior or eval can be weak
Ed25519 receipt	the evaluator can be wrong or compromised
blast-radius check	in-scope changes can still be harmful
environment receipt	structural proof may not equal deployed behavior
human receipt	the human can make a bad decision

WHY THIS EARNS TRUST A system that names what it cannot prove is auditable. The closing promise is **inspectable authority and proof – not guaranteed success.**

Task-Spec authorizes. TaskMesh coordinates.

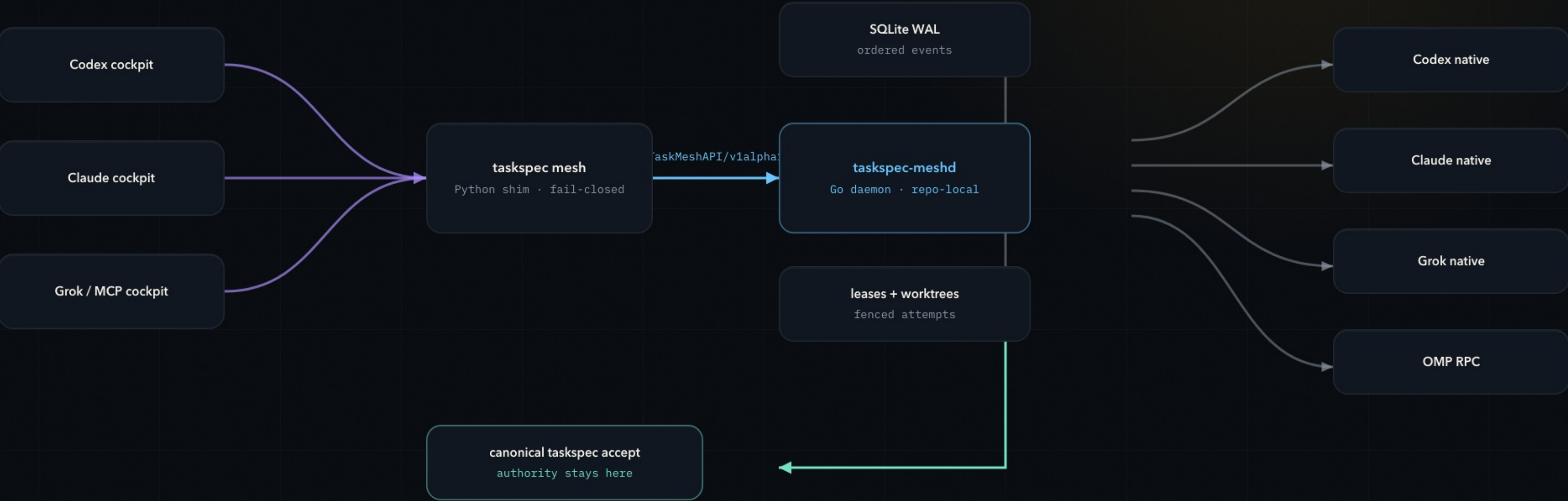
TaskMesh consumes the canonical Tier-1 frontier; it cannot widen scope, rewrite a signed leaf, or accept on its own terms. **Optional means removable.**



TASK-SPEC	TASKMESH
canonical task authority	optional execution control plane
graph and ready frontier	leases and waves
PRE and POST	routes and observes attempts
acceptance record	worktrees and integration branch
human topology / target merge	no target-branch mutation

The local control plane. One daemon, many cockpits.

Multiple cockpits share one durable, repository-local daemon and event history. The Python shim exists so the optional helper can **fail closed before any run**.



SOURCES · [I] src/meshctl/cli.py · src/meshctl/mcp_server.py · mesh/cmd/taskspec-meshd/ · mesh/internal/mesh/

Exact version negotiation comes first.

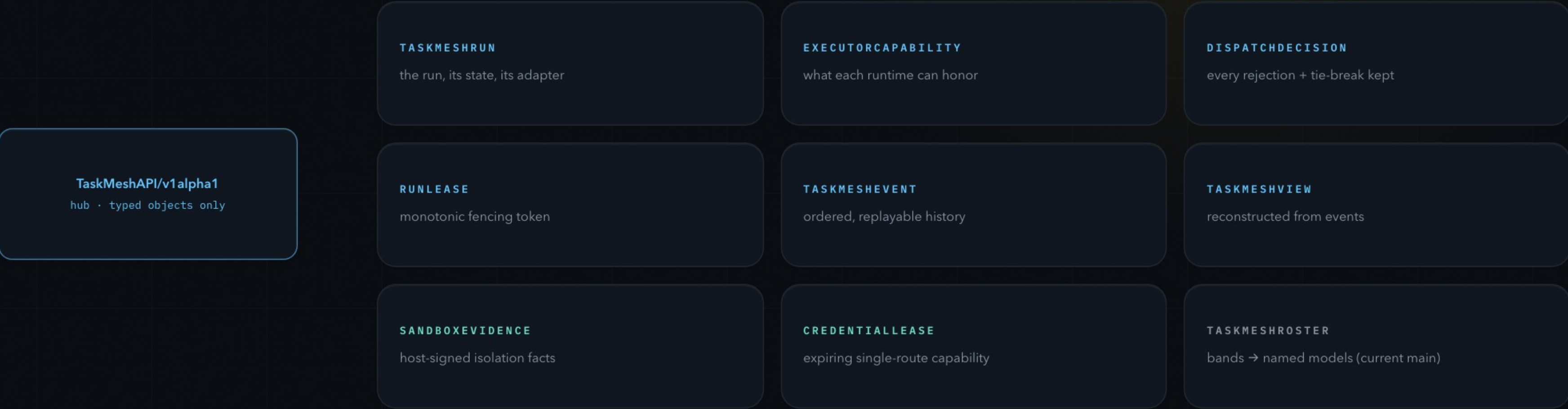
Core-only installs explain how to add TaskMesh; mismatched helpers return a stable runtime error. **No silent drift between CLI and daemon.**



SOURCES · [I] `src/meshctl/cli.py` · `mesh/cmd/taskspec-meshd/main.go` · [E] `tests/test-mesh-contracts.sh`

TaskMesh speaks versioned contracts.

Run, capability, route, lease, event, view, sandbox, credential, and roster objects are **machine-readable** – the control plane is inspectable and reconnectable.



POINT those objects describe a run that survives the cockpit – disconnect, reconnect, same truth.

SOURCES · [N] spec/schemas/taskmesh-*.schema.json · [D] docs/reference/taskmesh-contracts.md

Durable history lives with the repository.

SQLite WAL + ordered events rebuild the same view after client or daemon restart. **Replay, not memory.**



cockpit A – before crash

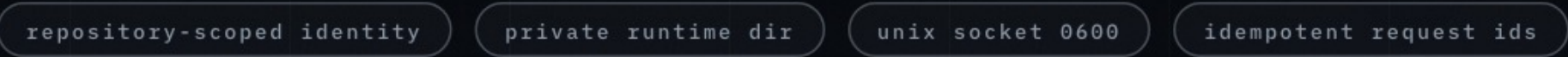
view

```
run: run-42 state: running lease: f=7
task: T-leaf adapter: codex-native
```

cockpit B – after reconnect

view

```
run: run-42 state: running lease: f=8
task: T-leaf adapter: codex-native
```



CONTINUITY Any cockpit can reconnect and see identical run IDs and state – because nothing lives in a client’s memory. The daemon is repository-local infrastructure, **not a hosted control plane.**

SOURCES · [I] mesh/internal/mesh/store.go · daemon.go · repository.go · [E] tests/test-mesh-daemon.sh · tests/test-mesh-cockpit.sh

An attempt advances through explicit states.

Execution, verification, supervision, acceptance, and integration are **separate transitions** – each one observable in the event log.



BOUNDARY an attempt state is **not** Task-Spec lifecycle authority. Canonical acceptance remains the gate; **integrated** still required a receipt.

A wave contains only authorized, disjoint leaves.

Only current Tier-1, dependency-ready, non-conflicting leaves are eligible. The frontier passes five filters; the first concurrency group emerges as parallel lanes.

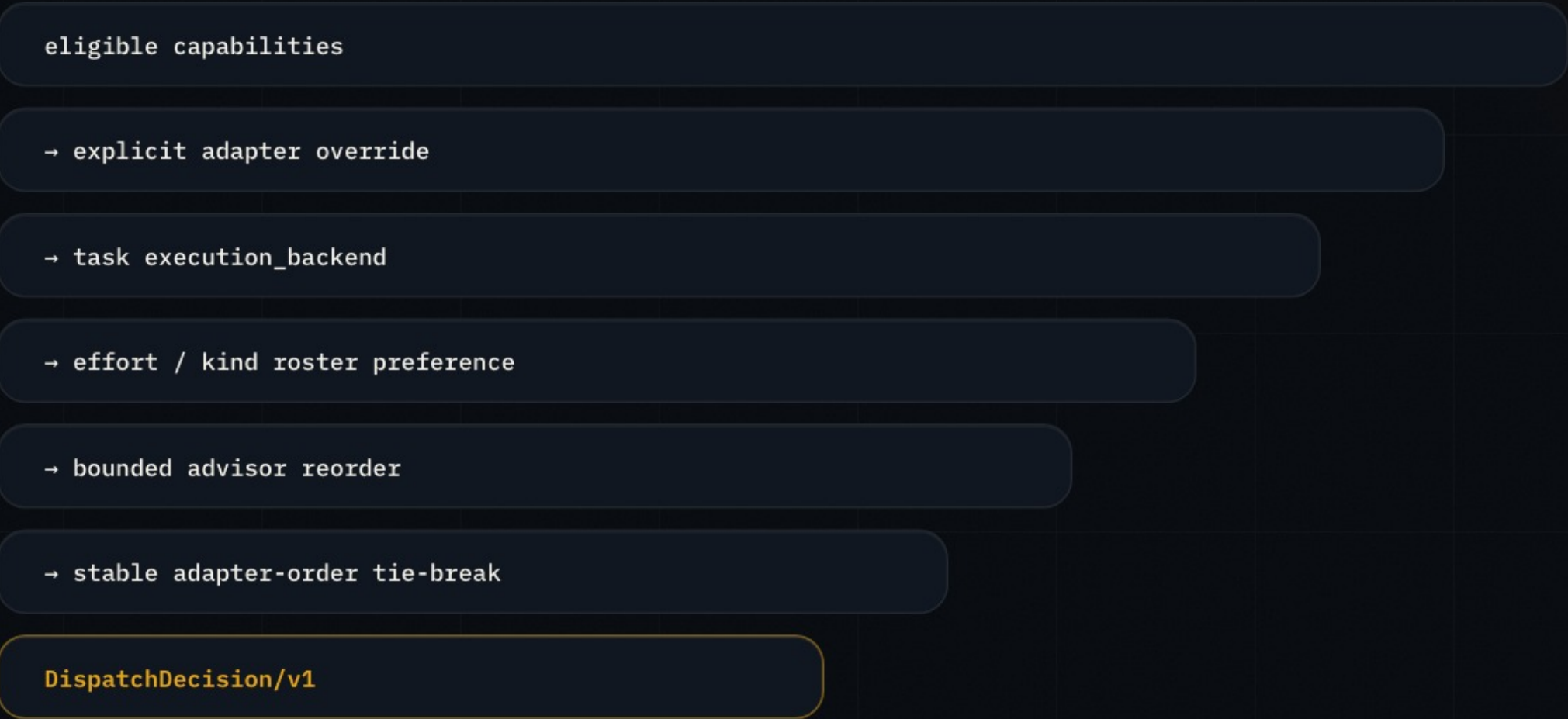


WAVE LAW Eligibility narrows the set; routing chooses one executor per leaf **without expanding the set**. A wave is a theorem the graph proves, not a scheduler’s hope.

SOURCES · [I] mesh/internal/mesh/graph.go · routing.go · [E] tests/test-mesh-leases.sh

Routing may rank. It may not invent.

Every candidate, rejection, preference, advisor reorder, and tie-break is **retained in DispatchDecision/v1** – explainability is a contract, not a log level.



RED BOUNDARY

The advisor can reorder eligible adapters; it **cannot add an unknown or ineligible adapter**. Ranking inside the set is allowed; growing the set is not.

AUDIT SHAPE

The decision object keeps the full funnel: candidates, rejections with reasons, preferences applied, and the final tie-break. “Why codex?” is always answerable.

Named-model rosters are current-main, not frozen 3.9 proof.

Current main maps effort bands and kinds to named models; `--model` wins and `require_named_model` fails closed. Labeled honestly: CURRENT MAIN

TaskMeshRoster/v1json

```
{
  "contract": "TaskMeshRoster/v1",
  "require_named_model": true,
  "bands": {
    "XS-S": { "candidates": [{ "adapter": "codex-native", "model": "..."}] },
    "M-L": { "candidates": [{ "adapter": "omp-rpc", "model": "..."}] }
  }
}
```

FAIL-CLOSED BY DEFAULT

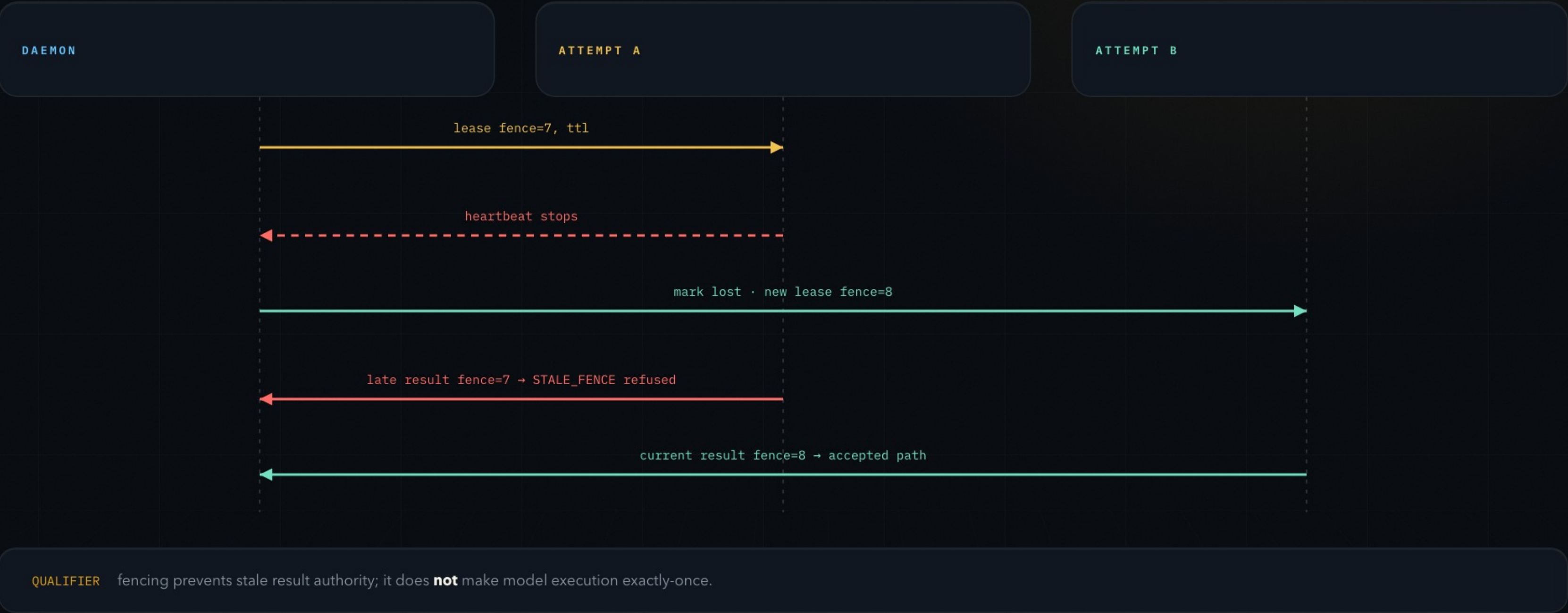
With **require_named_model**, a band without a named candidate refuses dispatch instead of improvising. **--model** overrides the roster explicitly, on the record.

HONEST GAP

The schema also carries `mode` and `failover`; at the audited commit they are parsed but **not consumed** by runtime routing. No semantic promise is made for them.

Leases prevent stale workers from regaining authority.

At most one active lease is authoritative per task revision; recovery issues a **higher fencing token**. Late results from dead attempts are refused, not merged.

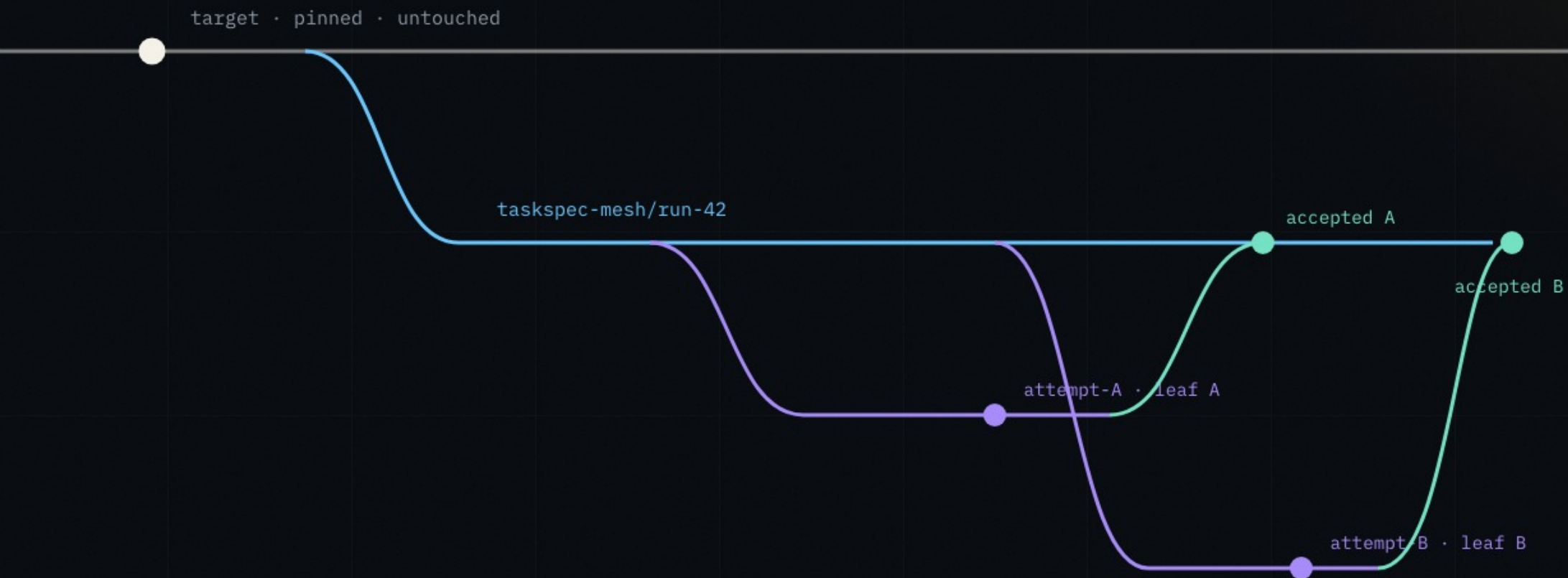


QUALIFIER fencing prevents stale result authority; it does **not** make model execution exactly-once.

SOURCES · [I] mesh/internal/mesh/lease.go · recovery.go · [E] tests/test-mesh-leases.sh

Worktrees isolate attempts from the target branch.

Accepted attempts merge into a **run integration branch**; the user's target branch remains untouched. Finish prints the human merge route – it never runs it.



HUMAN MERGE ROUTE
finish prints: check out target, merge run-42, resolve if needed. The last arrow is a person.
NO IMPLICIT REBASE

DIVERGENCE
target divergence or merge conflict → park safely. Never force, never silently rebase.

Supervised execution is process separation, not a sandbox.

Native Codex, Claude, Grok, and OMP adapters preserve **one handoff and one observable attempt**. Local process controls do not create hostile-code isolation – and say so.

CODEX NATIVE workspace-write · ephemeral supervised; sandbox-profile limited; no persistence beyond the attempt.	CLAUDE NATIVE acceptEdits mode edits accepted inside the worktree only; tool surface bounded per attempt.	GROK NATIVE subagents/memory/web off attack surface reduced by disabling autonomous extras during the attempt.	OMP RPC supervised session timeouts, cancellation, bounded output, redaction – observable, not isolated.
--	---	--	--

one TaskHandoff/v3 rail · four process shapes · one acceptance gate

BOUNDARY timeouts, cancellation, bounded output, and redaction are **process hygiene**. They are not a claim about hostile code – the next slide shows what that claim actually requires.

Autonomous OMP fails closed without isolation.

Autonomous mode requires an explicitly verified container, route, model, capability, and **host-signed evidence chain**. Missing proof → refusal, never downgrade.

SURFACE	SUPERVISED	AUTONOMOUS OMP
execution	local process adapter	non-root container worker
supervision	mandatory for native adapters	not interactive after verified preflight
credentials	host process environment	expiring attempt capability via proxy
filesystem	attempt worktree	exactly one writable workspace mount
isolation claim	none against hostile code	bounded controls + host evidence
downgrade	n/a	forbidden; missing proof fails closed

- missing runtime
- unverified proxy
- image digest mismatch
- missing model
- changed evidence

SOURCES · [I] mesh/internal/mesh/sandbox.go · credential.go · release/mesh/Dockerfile · release/mesh/image.lock · [E] tests/test-mesh-isolation.sh

Credentials stay outside the worker.

The worker receives **one expiring capability for one provider route** – not the host’s credential set. The most important isolation detail is what the worker never receives.

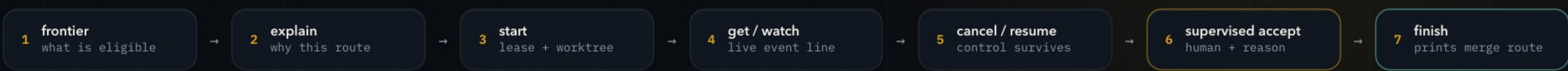


LEAST AUTHORITY Capability expiry, route pinning, and redaction make the worker’s reach **finite by construction** – and the cockpit can disappear because authority and history live elsewhere.

SOURCES · [I] mesh/internal/mesh/credential.go · sandbox.go · release/mesh/Dockerfile · [N] credential/sandbox schemas

Any cockpit can reconnect; only a human finishes the target merge.

CLI and local MCP expose the same durable run. The Codex cockpit can disconnect; the Claude cockpit reconnects to the **same event line**. Finish prints merge commands – not an automatic merge.



mesh finish – output

```
# run-42 accepted · integration branch taskspec-mesh/run-42
git checkout main
git merge --no-ff taskspec-mesh/run-42  # you run this · not the mesh
taskspec status T-leaf                  # done · accepted: true
```

bash

MCP IS A COCKPIT, NOT A GATE

The local MCP server is a stateless window over the daemon. It cannot bypass leases, widen scope, or accept on its own terms – **same contracts, thinner glass**.

Forty-one commands, one lifecycle.

The surface is large because contracts, evidence, graph, runtime, and maintenance are explicit. The **everyday path stays short**: plan · batch · validate · dod · gate · handoff · accept.

prepare

init

setup

setup signing

doctor

demo

example

author

new

plan

batch

migrate

author-doctor

PRE

validate

dod

gate

handoff

execute / POST

run

accept

evidence / interop

holdout

receipt

eval-audit

identity

evidence

bridge

dsse

mcp

graph / backlog

ready

graph

status

lint

transition

rebuild-state

archive

backup

metrics

runtime / support

mesh

conformance

executor

agent-context

completion

version

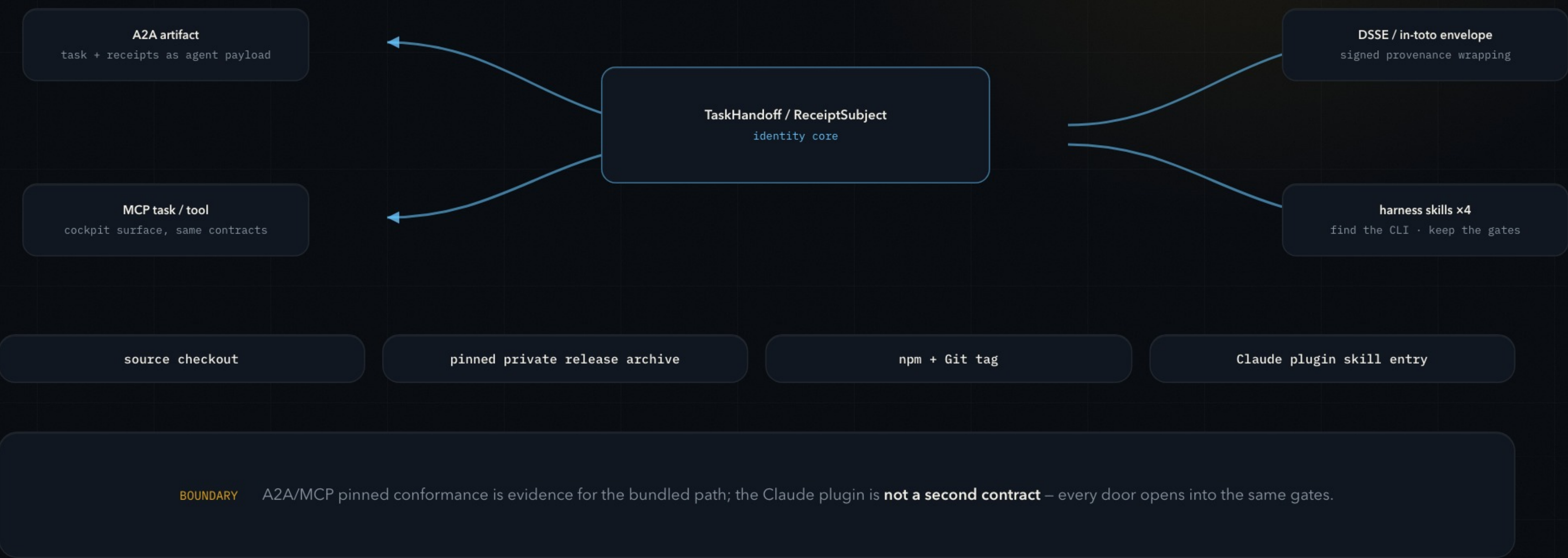
help

EXIT CODES

global --json · --dry-run · 0 success · 1 contract/gate failure · 2 usage · 3 runtime floor

Portable outside the CLI, too. Identity survives the bridge.

A2A, MCP, DSSE, skills, and installers preserve contract identity while staying honest: pinned conformance is **evidence, not ecosystem-wide certification**.



Task-Spec dogfoods Task-Spec.

Approved release manifests decomposed 3.8.1 and TaskMesh 3.9.0 into **accepted leaves and parked composition nodes** – methodology as repository evidence.



accepted leaf – sanitized receipt

md

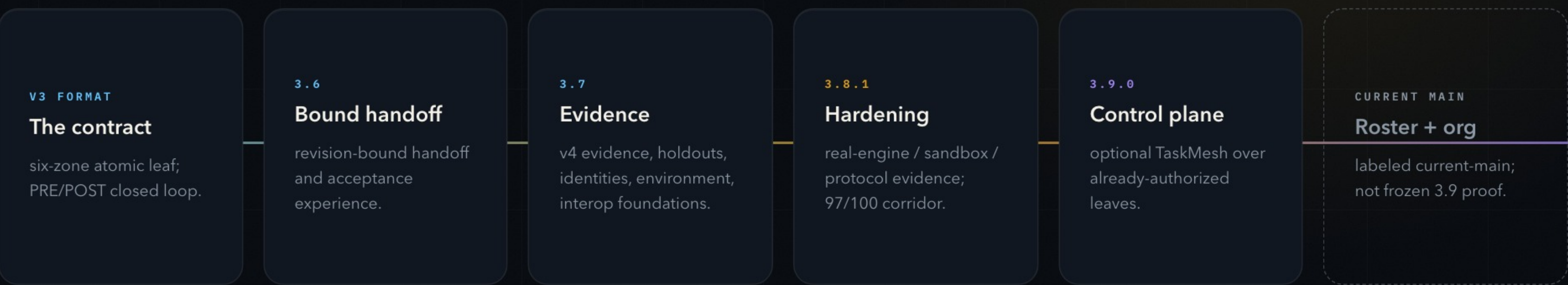
```
signed_off: true    tier: 1
authorization: hmac-sha256-v3:...
attempt_id: uuid
accepted: true    acceptance_record: sha256:...
```

WHAT THIS PROVES

The release program ran on its own contracts: leaves were authorized, attempted, and accepted through the real gates; composition nodes parked honestly instead of faking completion.

The evolution is a trust-chain story.

Releases add explicit trust surfaces while preserving readable older formats. Five milestones, not every patch: each one tightened **one proof boundary at a time**.



PATTERN every release answered one question: “what did we make inspectable that used to be implicit?”

Read the evidence in two corridors.

The 3.8.1 quality score and the 3.9.0 TaskMesh proof answer **different questions**. Neither is collapsed into the other; current-main drift is labeled, not hidden.

3.8.1 · QUALITY CORRIDOR

97/100 under a fixed rubric

Authority, lifecycle, docs, packaging, engines, protocol pins, signed sandbox. Three points deliberately unavailable: **semantic truth, ecosystem certification, production reliability**.

release/3.8.1/scorecard.json

3.9.0 · TASKMESH CORRIDOR

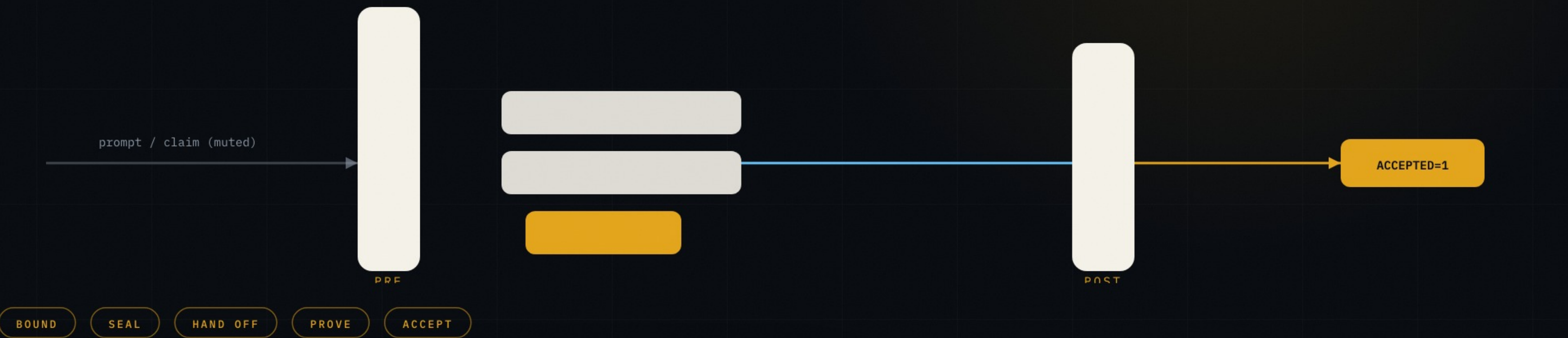
Control-plane evidence retained

Install, live isolation, recovery, cockpit, hosted conformance, private provenance, publication. No hosted control plane, no auto target merge, no Tier-1-from-worktrees.

release/3.9.0/mesh-release-evidence.json

CURRENT MAIN most exact make check suites revalidated 2026-08-26; final gate currently fails on missing README release-status markers – a documentation projection drift, named on the record.

Make the agent earn done.



```
reviewer route bash

taskspec demo
taskspec example task-plan --out /tmp/task-plan.yaml
taskspec plan --manifest /tmp/task-plan.yaml
make check
python3 src/evidence/release_audit.py audit
```

HONEST STATUS

At the audited commit, make check reaches the release-status projection drift named on slide 65. Do not present the route as all-green until markers are corrected and rerun.

“The worker may propose, execute, retry, and report. The contract decides what was authorized. The receipt decides what was accepted.”

SOURCES · [D] docs/getting-started/reviewer-route.md · [I] src/setup/demo.sh · [E/C] release evidence and current audit